Tailwind CSS Notes

Based on your crash course and practice project (navbar, dark mode toggle, responsive feature grid). Covers spacing, flexbox, grid, hover effects and custom colors, plus the extra topics you need to make the notes complete.

1. What Tailwind is

Tailwind is a utility-first CSS framework. Instead of writing CSS in a separate file, you style elements by adding small single-purpose classes straight in your markup (`className` in React).

```
<div className="bg-slate-500 p-4 rounded text-white">Hello</div>
```

Key ideas:

- Each class does one thing (`p-4` = padding, `rounded` = border radius).
- You combine classes to build a design.
- Only the classes you actually use end up in the final CSS, so the output is small.
- Responsive design, hover states and dark mode are handled with **prefixes** (`sm:` , `hover:` , `dark:`).

2. Spacing

Tailwind uses a spacing scale where **1 unit = 0.25rem = 4px**.

Class	rem	px
`1`	0.25rem	4px
`2`	0.5rem	8px
`4`	1rem	16px
`6`	1.5rem	24px
`8`	2rem	32px
`12`	3rem	48px
`16`	4rem	64px

Padding (inside the element)

Class	Meaning
`p-4`	padding on all sides
`px-4`	left and right
`py-4`	top and bottom
`pt-4` `pr-4` `pb-4` `pl-4`	top, right, bottom, left

Margin (outside the element)

Same pattern with `m`: `m-4`, `mx-4`, `my-4`, `mt-4`, `mb-4`, `ml-4`, `mr-4`.

- `mx-auto` centers a block element horizontally (it needs a width, e.g. `max-w-md`).
- Negative margins use a dash: `-mt-4`.

Gap and space-between

- `gap-4` spaces children inside a flex or grid container. Use `gap-x-4` and `gap-y-4` for one direction.
- `space-x-4` / `space-y-4` add margin between children (older approach, `gap` is usually better).

Width and height

- Scale: `w-4`, `w-64`, `h-10`
- Fractions: `w-1/2`, `w-1/3`, `w-full`
- Viewport: `w-screen`, `h-screen`, `min-h-screen`
- Limits: `max-w-md`, `max-w-xl`, `min-w-0`
- `size-10` sets width and height together.

From your project

```
<div className="flex items-center justify-between p-4">   // padding all
sides
<div className="grid ... gap-6 p-6">                        // gap between
cards + padding
<div className="bg-slate-500 p-4 rounded">                 // padding inside
each card
```

3. Flexbox

Flexbox lays items out in **one dimension** (a row or a column).

Turning it on

- `flex` makes the element a flex container (children sit in a row by default).
- `inline-flex` is the inline version.

Direction

Class	Effect
`flex-row`	left to right (default)
`flex-col`	top to bottom
`flex-row-reverse` / `flex-col-reverse`	reversed

Main axis: `justify-*`

Controls alignment along the direction of flow (horizontal in a row).

- `justify-start`, `justify-center`, `justify-end`
- `justify-between`: first item at start, last at end, space between the rest
- `justify-around`, `justify-evenly`

Cross axis: `items-*`

Controls alignment perpendicular to the flow (vertical in a row).

- `items-start`, `items-center`, `items-end`, `items-stretch` (default), `items-baseline`

Wrapping and sizing children

- `flex-wrap`, `flex-nowrap`
- `flex-1`: item grows to fill available space
- `grow` / `shrink-0`: allow growing / stop shrinking
- `basis-1/2`: starting size
- `self-center`, `self-end`: override alignment for one child

Centering anything (very common)

```
<div className="flex items-center justify-center h-screen">
  <p>Perfectly centered</p>
</div>
```

From your project

```
// Navbar: logo on the left, links on the right, vertically aligned
<div className="flex items-center justify-between p-4">

// Desktop links in a row with a small gap
<div className="hidden sm:flex gap-2">

// Mobile menu: stacked in a column, centered
<div className="flex flex-col items-center gap-2 p-4 sm:hidden">
```

Note how `hidden sm:flex` means: hidden by default, becomes `flex` from the `sm` breakpoint up.

4. Grid

Grid lays items out in **two dimensions** (rows and columns). Use it for card layouts and page sections.

Basics

- `grid` turns it on.
- `grid-cols-3` makes 3 equal columns, and `grid-rows-2` makes 2 rows.
- `gap-6` sets spacing between cells.

Spanning

- `col-span-2` : item takes 2 columns
- `row-span-2` : item takes 2 rows
- `col-span-full` : item spans every column

Flexible columns

- `grid-cols-[200px_1fr]` : fixed sidebar plus flexible main (arbitrary value)
- `grid-cols-[repeat(auto-fit,minmax(200px,1fr))]` : auto-responsive without breakpoints

Alignment

- `place-items-center` centers content inside every cell.
- `justify-items-*` , `items-*` work like flex.

Responsive grid (your project)

```
<div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 gap-6 p-6">
```

Read it mobile first:

- Default (phones): 1 column
- `sm` (640px+): 2 columns
- `md` (768px+): 3 columns

Flex vs Grid

Use flex when	Use grid when
Navbars, button rows, centering	Card galleries, page layouts
One direction at a time	Rows and columns together
Content decides the size	Layout decides the size

5. Responsive design (breakpoints)

Tailwind is **mobile first**: classes without a prefix apply to all screens, and prefixed classes apply from that breakpoint **upwards**.

Prefix	Min width
`sm:`	640px
`md:`	768px
`lg:`	1024px
`xl:`	1280px
`2xl:`	1536px

```
<div className="text-sm md:text-base lg:text-lg">
```

This means small text on phones, base on tablets, large on laptops.

Important: write the mobile style first, then override upwards. Do not think of `sm:` as meaning phones. It means "640px and bigger".

Show and hide

- `hidden`: `display: none`
- `sm:hidden`: hide from `sm` and up (your hamburger button)
- `hidden sm:flex`: hidden on phones, flex from `sm` up (your desktop nav)

6. Hover effects and other states

Prefix any utility with a state to apply it only in that state.

```
<div className="bg-slate-500 hover:bg-slate-600">
```

Common state prefixes

Prefix	When it applies
`hover:`	mouse over
`focus:`	element focused (inputs, buttons)
`focus-visible:`	focused by keyboard
`active:`	while being pressed
`disabled:`	disabled element
`first:` `last:` `odd:` `even:`	position among siblings
`group-hover:`	when a parent marked `group` is hovered
`peer-checked:`	based on a sibling marked `peer`

Transitions (make hover smooth)

Without a transition, hover changes snap instantly.

- `transition` or `transition-all`: which properties animate
- `duration-300`: how long (ms)
- `ease-in-out`: easing curve
- `delay-100`: wait before starting

Transforms

- `scale-105`: grow 5%
- `rotate-6`, `-rotate-6`
- `translate-x-2`, `-translate-y-1`
- `hover:scale-105`, `hover:-translate-y-1`

From your project

```
<div className="bg-slate-500 p-4 rounded hover:bg-slate-600 hover:scale-105
transition-all duration-300">
  Feature one
</div>
```

This card darkens and grows slightly on hover, animated over 300ms.

Group hover example

```
<div className="group p-4">
  <h3>Title</h3>
  <p className="opacity-0 group-hover:opacity-100 transition">Shown on
hover</p>
</div>
```

Other useful hover bits

- `cursor-pointer` shows the hand cursor (you used it on buttons)
- `hover:underline`, `hover:text-blue-500`, `hover:shadow-lg`

7. Colors

Built-in palette

Format: `{property}-{color}-{shade}`

- Properties: `bg-`, `text-`, `border-`, `ring-`, `divide-`, `from-` / `to-` (gradients)
- Colors: `slate`, `gray`, `red`, `blue`, `green`, `yellow`, `purple`, `pink`, and more
- Shades: `50` (lightest) up to `950` (darkest)

```
<div className="bg-slate-900 text-white border border-slate-700">
```

Opacity

- `bg-black/50` is black at 50% opacity
- `text-white/80`
- Separate from `opacity-50`, which fades the whole element

Custom colors, Method A: arbitrary values (one-off)

Use square brackets for any value:

```
<div className="bg-[#1da1f2] text-[#fff] border-[rgb(10,20,30)]">
```

Good for quick tests. Not great if you reuse the color often.

Custom colors, Method B: Tailwind v4 (`@theme` in CSS)

In v4 you define design tokens in your main CSS file. There is no `tailwind.config.js` required.

```
/* index.css */
@import "tailwindcss";

@theme {
  --color-brand: #4f46e5;
  --color-brand-light: #818cf8;
  --color-brand-dark: #3730a3;
}
```

Now these classes exist automatically:

```
<button className="bg-brand hover:bg-brand-dark text-white">Click</button>
<p className="text-brand-light">Hello</p>
```

The name after `--color-` becomes the class name.

Custom colors, Method C: Tailwind v3 (`tailwind.config.js`)

```
// tailwind.config.js
export default {
  content: ["./index.html", "./src/**/*.{js,jsx,ts,tsx}"],
  theme: {
    extend: {
      colors: {
        brand: {
          DEFAULT: "#4f46e5",
          light: "#818cf8",
          dark: "#3730a3",
        },
      },
    },
  },
};
```

Usage: `bg-brand`, `bg-brand-light`, `text-brand-dark`.

Tip: use `extend` so you add colors without deleting the default palette. Putting colors directly under `theme` (without `extend`) replaces everything.

How to know which version you have

- v4: you see `@import "tailwindcss";` in CSS and usually `@tailwindcss/vite` in the Vite config.
- v3: you see `@tailwind base; @tailwind components; @tailwind utilities;` and a `tailwind.config.js`.

8. Dark mode

Use the `dark:` prefix to style elements when dark mode is active.

```
<div className="bg-white text-slate-900 dark:bg-slate-900 dark:text-white">
```

Two strategies

1. **Media (default):** follows the user's operating system setting. No code needed.
2. **Class (manual toggle):** dark mode turns on when a `dark` class exists on a parent. This is what your project uses with the toggle button.

Enabling class-based dark mode

Tailwind v4 (add to your CSS):

```
@import "tailwindcss";
@custom-variant dark (&:where(.dark, .dark *));
```

Tailwind v3 (config file):

```
export default {
  darkMode: "class",
  // ...
};
```

Your toggle logic

```
const [theme, setTheme] = useState("light");

<div className={`${theme} min-h-screen dark:bg-slate-900 dark:text-white`}>
  <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
```

The `theme` state is either `"light"` or `"dark"` and is injected as a class name. When it is `"dark"`, every `dark:` utility inside that div activates.

Important note about your project

If you did not add the `@custom-variant` line (v4) or `darkMode: "class"` (v3), then `dark:` will follow the system theme and **your button will not work** as expected. Also note that your root div has no light-mode background (e.g. `bg-white`), so add one for a consistent look:

```
className={`${theme} min-h-screen bg-white text-slate-900 dark:bg-slate-900
dark:text-white`}
```

Persisting the choice: save `theme` to `localStorage` and read it on load, so it survives a refresh.

9. Typography

Category	Classes
Size	`text-xs` `text-sm` `text-base` `text-lg` `text-xl` `text-2xl` … `text-9xl`
Weight	`font-light` `font-normal` `font-medium` `font-semibold` `font-bold`
Align	`text-left` `text-center` `text-right`
Spacing	`tracking-wide` (letter), `leading-6` (line height)
Style	`italic`, `underline`, `uppercase`, `capitalize`, `truncate`
Family	`font-sans`, `font-serif`, `font-mono`

Your project uses `font-semibold`, `text-sm md:text-base`, `text-center`, and `font-bold`.

10. Borders, radius, shadows

- **Border:** `border`, `border-2`, `border-t`, `border-slate-300`
- **Radius:** `rounded`, `rounded-md`, `rounded-lg`, `rounded-xl`, `rounded-full` (circles and pills)
- **Shadow:** `shadow`, `shadow-md`, `shadow-lg`, `shadow-xl`
- **Ring (outline-like):** `ring-2 ring-blue-500`, often used with `focus:`
- **Divide:** `divide-y` adds lines between children

11. Display and positioning

- **Display:** `block`, `inline`, `inline-block`, `flex`, `grid`, `hidden`
- **Position:** `static`, `relative`, `absolute`, `fixed`, `sticky`
- **Offsets:** `top-0`, `right-4`, `inset-0` (all sides)
- **Layering:** `z-10`, `z-50`
- **Overflow:** `overflow-hidden`, `overflow-auto`, `overflow-x-scroll`

Common patterns:

```
// Sticky navbar
<nav className="sticky top-0 z-50 bg-white">

// Badge in the corner of a card
<div className="relative">
  <span className="absolute top-2 right-2">New</span>
</div>
```

12. Backgrounds and gradients

- Solid: `bg-blue-500`
- Gradient: `bg-gradient-to-r from-purple-500 to-pink-500` (v3 and v4; v4 also supports `bg-linear-to-r`)
- Via color: `from-blue-500 via-purple-500 to-pink-500`
- Image: `bg-[url('/hero.jpg')] bg-cover bg-center`

13. Arbitrary values

When the scale does not have what you need, use square brackets:

```
<div className="w-[350px] mt-[13px] bg-[#0f172a] text-[18px] grid-cols-
[1fr_2fr]">
```

Use sparingly. If you repeat the same value, define it in your theme instead.

14. Setup (Vite + React)

Tailwind v4:

```
npm create vite@latest my-app -- --template react
cd my-app
npm install tailwindcss @tailwindcss/vite
```

```
// vite.config.js
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

```
/* src/index.css */
@import "tailwindcss";
```

Import `index.css` in `main.jsx`, then run `npm run dev`.

Tailwind v3 uses `npm install -D tailwindcss postcss autoprefixer`, then `npx tailwindcss init -p`, a `content` array in the config and the three `@tailwind` directives in CSS.

Tip: install the **Tailwind CSS IntelliSense** VS Code extension for autocomplete.

15. Dynamic classes in React (common pitfalls)

Never build class names by string concatenation

Tailwind scans your source for complete class names. This will **not** work:

```
// Wrong
<div className={`bg-${color}-500`}>
```

This works, because the full names appear in the code:

```
// Right
const colors = { red: "bg-red-500", blue: "bg-blue-500" };
<div className={colors[color]}>
```

Conditional classes

```
<button className={`px-4 py-2 rounded ${active ? "bg-blue-500 text-white" :
"bg-gray-200"}`}>
```

For bigger projects use the `clsx` library, which gives cleaner conditionals.

Remember

- In React it is `className`, not `class`.
- Class order does not matter for the result. Use the Prettier Tailwind plugin to auto-sort them.
- Conflicting classes on the same element (e.g. `p-2 p-4`) are decided by CSS order in the stylesheet, not by their order in your string, so avoid them.

16. Reusing styles

When the same long class list repeats (like your six identical feature cards):

1. **Best: make a React component, or map over data.**

```
const features = ["one", "two", "three", "four", "five", "six"];

{features.map((f) => (
  <div key={f} className="bg-slate-500 p-4 rounded hover:bg-slate-600
hover:scale-105 transition-all duration-300">
    Feature {f}
  </div>
))}
```

2. **Occasionally: `@apply` in CSS.**

```
.card {
  @apply bg-slate-500 p-4 rounded hover:bg-slate-600;
}
```

Prefer components over `@apply`, since that is the style Tailwind recommends.

17. Review of your practice project

Part of the code	Concept it shows
`min-h-screen`	Full viewport height minimum
`flex items-center justify-between p-4`	Flex navbar with spacing
`hidden sm:flex gap-2` / `sm:hidden`	Responsive show and hide

Part of the code	Concept it shows
`{open && (...)}`	React conditional rendering for the mobile menu
`flex flex-col items-center gap-2`	Vertical stacked menu
`grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 gap-6`	Responsive grid
`hover:bg-slate-600 hover:scale-105 transition-all duration-300`	Smooth hover effect
`dark:bg-slate-900 dark:text-white`	Dark mode
`text-sm md:text-base`	Responsive typography

Small improvements you could make

- Add `bg-white` for light mode on the root div.
- Make sure class-based dark mode is enabled in your CSS or config, as in section 8.
- Replace the six repeated cards with a `.map()`.
- Use semantic HTML: `<nav>`, `<a href>` instead of `<span>`, and add `aria-label="Toggle menu"` to icon buttons.
- Add `transition-colors` on the root so the theme change fades.

18. Quick cheat sheet

I want to...	Use
Add inner space	`p-4`, `px-6`
Add outer space	`m-4`, `mx-auto`
Space between children	`gap-4`
Row layout	`flex`
Column layout	`flex flex-col`
Center everything	`flex items-center justify-center`
Push items apart	`justify-between`

I want to...	Use
Card grid	`grid grid-cols-3 gap-6`
Responsive columns	`grid-cols-1 md:grid-cols-3`
Hide on mobile	`hidden md:block`
Hover color	`hover:bg-blue-600`
Smooth change	`transition duration-300`
Grow on hover	`hover:scale-105`
Dark mode style	`dark:bg-slate-900`
Custom color	`bg-[#123456]` or define in `@theme`
Rounded corners	`rounded-lg`
Shadow	`shadow-md`
Full height	`min-h-screen`

19. Practice ideas

1. Build a pricing section with 3 cards, where the middle one is highlighted and larger.
2. Add a hero section with a gradient background and a centered call-to-action button.
3. Define `brand` colors with `@theme` and use them across the page.
4. Make a footer with a 4-column grid that collapses to 1 column on phones.
5. Save the theme to `localStorage` so the dark mode choice persists.